C++ Standard Library Issues Resolved Directly In Toronto

Doc. no. P0699R0

Date: Revised 2017-07-14 at 21:21:37 UTC

Project: Programming Language C++
Reply to: Marshall Clow <lwgchair@gmail.com>

Immediate Issues

2901. Variants cannot properly support allocators

Section: 23.7.3 [variant.variant] **Status:** Immediate **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-07-14

Priority: 0

View other active issues in [variant.variant].

View all other issues in [variant.variant].

View all issues with Immediate status.

Discussion:

Addresses US 113

Variants cannot properly support allocators, as any assignment of a subsequent value throws away the allocator used at construction. This is not an easy problem to solve, so `variant` would be better served dropping the illusion of allocator support for now, leaving open the possibility to provide proper support once the problems are fully understood.

Proposed change: Strike the 8 allocator aware constructor overloads from the class definition, and strike 20.7.2.1 [variant.ctor] p34/35. Strike clause 20.7.12 [variant.traits]. Strike the specialization of `uses_allocator` for `variant` in the `<variant>` header synopsis, 20.7.1 [variant.general].

[2017-02-28, Kona, Casey provides wording]

[2017-06-29 Moved to Tentatively Ready after 7 positive votes on c++std-lib.]

[2017-07 Toronto Moved to Immediate]

Proposed resolution:

This wording is relative to N4659.

1. Change 23.7.2 [variant.syn], header `<variant>` synopsis, as follows:

```
[…]
// 23.7.13 [variant.traits], allocator related traits
```

```
template <class T, class Alloc> struct uses_allocator;
template <class... Types, class Alloc> struct uses_allocator<variant<Types...>, Alloc>;
```

2. Change 23.7.3 [variant.variant], class template `variant` synopsis, as follows:

[…]

```
// allocator extended constructors
template <class Alloc>
  variant(allocator_arg_t, const Alloc&);
template <class Alloc>
  variant(allocator_arg_t, const Alloc&, const variant&);
template <class Alloc>
  variant(allocator_arg_t, const Alloc&, variant&&);
template <class Alloc, class T>
  variant(allocator_arg_t, const Alloc&, T&&);
template <class Alloc, class T, class... Args>
  variant(allocator_arg_t, const Alloc&, in_place_type_t<T>, Args&&...);
template <class Alloc, class T, class U, class... Args>
  variant(allocator_arg_t, const Alloc&, in_place_type_t<T>,
          initializer_list<U>, Args&&...);
template <class Alloc, size_t I, class... Args>
  variant(allocator_arg_t, const Alloc&, in_place_index_t<I>, Args&&...);
template <class Alloc, size_t I, class U, class... Args>
  variant(allocator_arg_t, const Alloc&, in_place_index_t<I>,
          initializer_list<U>, Args&&...);
```

3. Modify 23.7.3.1 [variant.ctor] as indicated:

```
// allocator extended constructors
template <class Alloc>
  variant(allocator_arg_t, const Alloc& a);
template <class Alloc>
  variant(allocator_arg_t, const Alloc& a, const variant& v);
template <class Alloc>
  variant(allocator_arg_t, const Alloc& a, variant&& v);
template <class Alloc, class T>
  variant(allocator_arg_t, const Alloc& a, T&& t);
template <class Alloc, class T, class... Args>
  variant(allocator_arg_t, const Alloc& a, in_place_type_t<T>, Args&&... args);
template <class Alloc, class T, class U, class... Args>
  variant(allocator_arg_t, const Alloc& a, in_place_type_t<T>,
          initializer_list<U> il, Args&&... args);
template <class Alloc, size_t I, class... Args>
  variant(allocator_arg_t, const Alloc& a, in_place_index_t<I>, Args&&... args);
template <class Alloc, size_t I, class U, class... Args>
  variant(allocator_arg_t, const Alloc& a, in_place_index_t<I>,
          initializer_list<U> il, Args&&... args);
```

-34- *Requires:* `Alloc` shall meet the requirements for an Allocator (20.5.3.5 [allocator.requirements]).

-35- *Effects:* Equivalent to the preceding constructors except that the contained value is constructed with uses-allocator construction (23.10.7.2 [allocator.uses.construction]).

4. Modify 23.7.13 [variant.traits] as indicated:

```
template <class... Types, class Alloc>
struct uses_allocator<variant<Types...>, Alloc> : true_type { };
```

-1- *Requires:* `Alloc` shall be an Allocator (20.5.3.5 [allocator.requirements]).

-2- [*Note:* Specialization of this trait informs other library components that variant can be constructed with an allocator, even though it does not have a nested

~~allocator_type. — *end note*]~~

2956. `filesystem::canonical()` still defined in terms of `absolute(p, base)`

Section: 30.10.15.2 [fs.op.canonical] **Status:** Immediate **Submitter:** Sergey Zubkov **Opened:** 2017-04-21 **Last modified:** 2017-07-14

Priority: 1

View all issues with Immediate status.

Discussion:

This is from editorial issue #1620:

Since the resolution of US-78 was applied (as part of P0492R2), `std::filesystem::absolute(path, base)` no longer exists. However, `std::filesystem::canonical` is still defined in terms of `absolute(p, base)`.

[2017-06-27 P1 after 5 positive votes on c++std-lib]

Davis Herring: This needs to be P1 — due to a wording gap in P0492R2, 2 out of the 3 overloads of `filesystem::canonical()` have bad signatures and are unimplementable.

[2017-07-14, Toronto, Davis Herring provides wording]

[2017-07-14, Toronto, Moved to Immediate]

Proposed resolution:

This wording is relative to N4659.

1. Edit 30.10.6 [fs.filesystem.syn] as indicated:

```
path canonical(const path& p~~, const path& base = current_path()~~);
path canonical(const path& p, error_code& ec);
~~path canonical(const path& p, const path& base, error_code& ec);~~
```

2. Edit 30.10.15.2 [fs.op.canonical] as indicated:

```
path canonical(const path& p~~, const path& base = current_path()~~);
path canonical(const path& p, error_code& ec);
~~path canonical(const path& p, const path& base, error_code& ec);~~
```

-1- *Effects:* Converts `p`, which must exist, to an absolute path that has no symbolic link, *dot*, or *dot-dot* elements in its pathname in the generic format.

-2- *Returns:* A path that refers to the same file system object as `absolute(p~~, base~~)`. ~~For the overload without a `base` argument, `base` is `current_path()`. Signatures~~The signature with argument `ec` returns `path()` if an error occurs.

[…]